

BABEŞ–BOLYAI TUDOMÁNYEGYETEM KOLOZSVÁR
MATEMATIKA ÉS INFORMATIKA KAR
INFORMATIKA SZAK

Szakdolgozat

Automatikus Rendszámleolvasás YOLO és TrOCR segítségével



TÉMAVEZETŐ:

DR. SÁNDOR CSANÁD,
EGYETEMI ADJUNKTUS

SZERZŐ:

DÁCZ KRISZTIÁN

2026

BABEȘ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Diploma Thesis

Automatic Number Plate Recognition with YOLO and TrOCR



ADVISOR:

CSANÁD SÁNDOR, PhD
LECTURER

AUTHOR:

KRISZTIÁN DÁCZ

2026

UNIVERSITATEA BABEȘ–BOLYAI, CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

Lucrare de licență

Recunoașterea Automată a Plăcuțelor de Înmatriculare cu YOLO și TrOCR



CONDUCĂTOR ȘTIINȚIFIC:

DR. CSANÁD SÁNDOR,
LECTOR UNIVERSITAR

ABSOLVENT:

KRISZTIÁN DÁCZ

2026

BABEȘ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Diploma Thesis

Automatic Number Plate Recognition with YOLO and TrOCR

Abstract

Automatic Number Plate Recognition (ANPR) systems have gained significant attention in recent years due to their diverse applications in law enforcement, traffic management, parking systems, and surveillance. ANPR technology involves the detection, localization, and interpretation of license plate information from images or (live) video streams.

The scope of this thesis is to design and implement a real-time, embedded ANPR system capable of detecting and reading Romanian license plates from a live video stream, deployed on a Raspberry Pi 5.

The solution presented in this thesis consists of two main steps: first, we detect and localize the license plate in the image using a yolov11n-obb object detection model, then we extract and recognize the characters of the detected plate using the TrOCR optical character recognition model.

We use a custom-trained deep learning model for license plate detection, leveraging oriented bounding boxes to handle plates at various angles and orientations. For the recognition phase, four OCR models were evaluated and compared: TrOCR-base, TrOCR-small, TesseractOCR, and EasyOCR, with TrOCR-small selected as the optimal solution based on accuracy and inference speed on the target hardware. This work is the result of my own activity. I have never provided or received unauthorised assistance in this work.

2026

KRISZTIÁN DÁCZ

ADVISOR:
CSANÁD SÁNDOR, PHD
LECTURER

Tartalomjegyzék

1. Bevezető	1
2. Elméleti háttér	3
2.1. Az ANPR rövid története	3
2.2. Gépi látás	4
2.3. Gépi tanulás	5
2.4. Optikai Karakterfelismerés	6
2.4.1. Az OCR technológiák fejlődése	7
2.4.2. Mi a TrOCR?	8
2.4.3. Miért a TrOCR?	9
2.4.4. Metrikák	10
2.5. Objektumdetektálás	10
2.5.1. Klasszikus objektumdetektálás	11
2.5.2. Mélytanulás alapú objektumdetektálás	12
2.5.3. Teljesítmény evaluálására szolgáló fő metrikák	13
2.5.4. Hatékonysági mutató - Mean Average Precision (mAP)	13
2.5.5. Mi az a YOLO?	14
2.5.6. Miért YOLOv11?	15
2.5.7. A YOLOv11 főbb jellemzői	15
2.5.8. A YOLOv11 architektúrája és rétegei	16
2.5.9. A YOLOv11 által támogatott fő számítógépes látási feladatok	18
3. Automatikus rendszámleolvasás Raspberry Pi-n	20
3.1. Használt programozási nyelv - Python	20
3.1.1. Miért ideális nyelv a Python mesterséges intelligencia projektekhez?	20
3.2. Adathalmaz	21
3.3. Használt eszköz: Raspberry Pi 5	24
3.3.1. Miért Raspberry Pi 5?	24
3.4. Használt mesterséges intelligencia modell: YOLOv11	25
3.4.1. YOLOv11 használata	25
3.4.2. YOLOv11 modellek összehasonlítása	26
3.4.3. YOLOv11-el elért tanítási eredmények	27
3.5. Használt karakterfelismerő modell: TrOCR	29
3.5.1. Melyik TrOCR verzió?	30

TARTALOMJEGYZÉK

3.6. Pipeline lépései a gyakorlatban	32
3.6.1. Rendszámok felismerése	32
3.6.2. Detektált rendszámok kivágása	33
3.6.3. Rendszámok olvasása	33
3.6.4. Olvasott rendszámok fájlba való logolása	33
Összefoglaló	34

1. fejezet

Bevezető

A technológiai fejlődés egyik legfőbb mozgatórugója az automatizálás, amelynek célja, hogy az emberi beavatkozást igénylő folyamatokat minimalizálja, ezáltal hatékonyabbá, gyorsabbá és megbízhatóbbá tegye. Az ipar, a logisztika, a mezőgazdaság és a közlekedés területén is megfigyelhető, hogy a manuális feladatokat egyre inkább mesterséges intelligencián alapszó rendszerek veszik át. Az automatizált megoldások nem csupán a munkaerőigényt csökkentik, hanem lehetőséget biztosítanak olyan valós idejű döntéshozatalra is, amely korábban elképzelhetetlen lett volna [14].

Ezen tendencia egyik gyakorlati példája az automatikus rendszámfelismerés (Automatic Number Plate Recognition - angolul ANPR), amely a közlekedési infrastruktúra digitalizációjának fontos eleme. Az ANPR rendszerek célja, hogy kameraképek alapján önállóan detektálják a járművek rendszámtábláit, majd azokat karakterfelismeréssel értelmezzék és feldolgozható adatformátumba alakítsák. Az ilyen rendszereket egyre szélesebb körben alkalmazzák - legyen szó parkolóautomatizálásról, forgalomfigyelésről, gyorsajtás ellenőrzéséről vagy biztonsági beléptetésről [11].

A szakdolgozat célja egy olyan automatikus rendszámfelismerő rendszer tervezése és megvalósítása, amely képes valós időben felismerni a járművek rendszámát egy videóforrás képkockáiból. A projekt keretében olyan számítógépes látásra (angolul Computer Vision) és gépi tanulásra (angolul Machine Learning) épülő megoldás kerül bemutatásra, amely a rendszámtábla lokalizálását, optikai karakterfelismerését (OCR), valamint az eredmények azonnali feldolgozását végzi el. A rendszer különös figyelmet fordít a pontosságra, a feldolgozási sebességre, valamint arra, hogy erőforrás-hatékony módon akár kis méretű, beágyazott eszközön - például Raspberry Pi-n - is működőképes legyen, ezáltal számos felhasználási területen alkalmazható. Telepíthető parkolók és garázsok beléptetőrendszereibe, alkalmazható magánterületek és ipari létesítmények megfigyelésére, közúti forgalomfigyelésre, ahol a rendszer rögzíti az áthaladó járművek rendszámait, illetve olyan rendőrségi alkalmazásokban is megállja a helyét, ahol lopott

1. FEJEZET: BEVEZETŐ

vagy körözött járművek azonosítása a cél.

Az ANPR rendszerek alapvető lépései, amelyekről egyenként fog beszámolni a dolgozat a későbbiekben, a következők [42]:

- Képkészítés
- Rendszám felismerés/lokalizálás
- Rendszám leolvasás

A dolgozat részletesen bemutatja a rendszámfelismeréshez kapcsolódó technológiai hátteret, a fejlesztés lépéseit, a használt eszközöket és algoritmusokat, valamint a megvalósított rendszer teljesítményének kiértékelését. A cél egy olyan, működő prototípus létrehozása, amely valós környezetben is demonstrálja a számítógépes látás gyakorlati alkalmazásának előnyeit, valós idejű felhasználás esetén.

Megemlítjük, hogy eme szakdolgozat során mesterséges intelligenciát használtunk, nyelvteni helyesség ellenőrzésére, szövegrészes újrafogalmazására, illetve az általunk előállított mérési adatokat tartalmazó .csv állományok táblázattá konvertálására.

2. fejezet

Elméleti háttér

Összefoglaló: Ez a fejezet a YOLO és TrOCR segítségével megvalósított automatikus rendszámfelismerés elméleti háttérét mutatja be. Először az ANPR technológia történeti fejlődését tekinti át az 1976-os kezdetektől napjainkig, kiemelve a klasszikus OCR-alapú megközelítésektől a mély tanuláson alapuló modern módszerekig vezető utat. Ezt követően ismerteti a gépi látás és a gépi tanulás alapfogalmait, különös tekintettel a felügyelt tanulásra, amely a dolgozatban alkalmazott modell tanítási módszerének alapja. A fejezet összefoglalja az optikai karakterfelismerés (OCR) technológiáit és fejlődési irányait, kitérve a dolgozatban használt TrOCR modell felépítésére és működési elvére. Végül, de nem utolsó sorban, részletesen tárgyalja az objektumdetektálás klasszikus és mély tanulás alapú megközelítéseit, bemutatja a rendszámok detektálására választott modell, a YOLOv11 architektúráját és főbb jellemzőit.

2.1. Az ANPR rövid története

Az ANPR (Automatic Number Plate Recognition), azaz az automatikus rendszámfelismerés, egy képfeldolgozási technológia, amely járművek rendszámtábláinak azonosítására szolgál. A rendszer a kamera által rögzített képből kinyeri a rendszám szövegét, és azt géppel olvasható formátumba, adatbázisba indexelhető szöveggé alakítja [27].

Kezdetek (1976–1990)

Az ANPR technológia gyökerei az Egyesült Királyságba nyúlnak vissza, az első rendszert 1976-ban fejlesztette ki a brit rendőrség kutatóintézete, a *Police Scientific Development Branch* [11]. A rendszer egy álló kamera által rögzített képeket, amiket analóg számítógéppel dolgozott fel, alapvető optikai karakterfelismerési módszerrel azonosítva a rendszámtábla karaktereit [4]. Az első működő prototípusok 1979-re készültek el, és kezdeti tesztelésük az A1-es úton, illetve a Dartford-alagútnál zajlott. Az első, ANPR-nek köszönhető letartóztatásra 1981-ben került sor, amikor a rendszer azonosított egy lopott járművet [20]. A korai rendszerek azonban még korlátozottak voltak: alacsony kameraminőség, korlátozott feldolgozási kapacitás

és a nem egységes rendszámformátumok mind nehezítették a pontos felismerést.

Fejlődés és szélesebb körű elterjedés (1990–2010)

A számítógépes látás és az optikai karakterfelismerés (OCR) fejlődésével az 1990-es években az ANPR rendszerek pontossága és sebessége jelentősen javult, olcsóbbá és könnyebben telepíthetővé váltak [11]. A technológia túllépett a rendvédelmi alkalmazásokon, és megjelent a díjfizetős kapuknál, parkolóknál, illetve határellenőrzési pontokon is. Ebben az időszakban három megközelítés dominált: a sablonillesztés (template matching), amely előre definiált karaktersablonokhoz hasonlítja a felismert alakzatokat, a hagyományos karakterfelismerési algoritmusok, amelyek morfológiai képfeldolgozással szegmentálták és azonosították a karaktereket, valamint a mesterséges neurális hálózatok, amelyek fokozatosan kiszorították a korábbi módszereket, robusztusságuknak, változó megvilágítási és szögelési viszonyok közötti jobb helytállásuknak köszönhetően [11].

Mély tanulás korszaka (2010–napjainkig)

A 2010-es évektől kezdve a mély tanulás, különösen a konvolúciós neurális hálózatok forradalmasították a rendszámfelismerést [27]. A hagyományos, lépésenkénti pipeline-t, amely külön képfeldolgozási, szegmentálási és OCR lépésekből állt, fokozatosan felváltották az end-to-end tanítható modellek, amelyek egyetlen hálózatban képesek elvégezni a teljes felismerési folyamatot.

2.2. Gépi látás

A gépi látás a mesterséges intelligencia egyik dinamikusan fejlődő ága, amelynek célja, hogy a gépek képesek legyenek képek és videók automatikus értelmezésére. A technológia lényege, hogy vizuális adatokból (például kameraképekből) strukturált, értelmezhető információt nyerjen ki, hasonló módon, mint ahogyan az emberi látás működik. Ehhez a gépi látás olyan módszereket alkalmaz, mint a mély tanulás (Deep Learning), konvolúciós neurális hálózatok (CNN-ek), valamint különféle képfeldolgozási algoritmusok.

A gépi látás kulcsszerepet tölt be az automatikus rendszámfelismerés (ANPR) működésében. A rendszer minden lépése, a jármű detektálásától a rendszám kiolvasásáig, vizuális információk feldolgozásán alapul, mind a gépi látás eszköztárát használják [12]:

2. FEJEZET: ELMÉLETI HÁTTÉR

- Objektumdetekció: A rendszámtáblák észlelése a kép különböző régióiban, gyakran mesterséges neurális hálózatok, például, ahogy e projekt esetében is, YOLO (You Only Look Once) segítségével történik.
- Képfeldolgozás: Kontrasztnövelés, zajszűrés, élesítés és binarizálás, annak érdekében, hogy a rendszámtábla karakterei jól elkülönüljenek a háttértől.
- Optikai Karakterfelismerés (OCR): Az optikai karakterfelismerés segítségével a detektált rendszámtábla karakterei digitális formában kiolvashatók.

A gépi látás előnye, hogy valós időben képes döntéseket hozni a vizuális adatok alapján, ezáltal alkalmazható valós idejű forgalomfigyelésre, beléptető rendszerek automatizálására, vagy akár rendőrségi használatra is. Ráadásul a modern algoritmusok már kis méretű, alacsony teljesítményű eszközökön is futtathatók, ezáltal akár egy Raspberry Pi-on is önálló rendszerek hozhatók létre.

A gépi látás tehát nem csupán lehetővé teszi a rendszámok felismerését, hanem egyúttal meg is alapozza az intelligens közlekedési rendszerek automatizált működését. A továbbiakban a gépi látás ezen alkalmazására kerül a fókusz, részletesen bemutatva az ehhez szükséges módszereket és technológiai hátteret.

2.3. Gépi tanulás

A gépi tanulás (angolul Machine Learning, röviden ML) a mesterséges intelligencia egyik kulcsfontosságú területe, amely lehetővé teszi, hogy a számítógépek emberi beavatkozás nélkül, tapasztalatokból, illetve adatokból tanuljanak és javítsák teljesítményüket egy adott feladat elvégzésében. A hagyományos programozással szemben, ahol a működési szabályokat előre pontosan meg kell adni, a gépi tanulás során a rendszer maga ismeri fel az összefüggéseket az adatokból. [10]

A gépi tanulás lényege tehát az, hogy minták alapján képes általánosítani, következtetéseket levonni és önálló döntéseket hozni. Ezt matematikai modellek és algoritmusok segítségével éri el, amelyek képesek az adatok jellemzőinek felismerésére, kategorizálására vagy predikciójára. Három fajtája van:

- Felügyelt tanulás (Supervised Learning): A rendszer egy előre címkézett adathalmazon tanul, azaz minden bemeneti adat mellé meg van adva a helyes kimenet (pl. képek és

2. FEJEZET: ELMÉLETI HÁTTÉR

azokhoz tartozó címkék). A cél, hogy új, ismeretlen adatokra is helyesen tudjon következtetni.

- Nem felügyelt tanulás (Unsupervised Learning): Itt az adatokhoz nem tartozik címke, a rendszer feladata az adatok közötti mintázatok, szerkezetek felismerése (pl. klaszterezés, anomália detekció).
- Megerősítéses tanulás (Reinforcement Learning): Egy ügynök interakcióba lép a környezetével, és úgy tanul, hogy jutalmakat vagy büntetéseket kap a cselekedetei alapján.

A dolgozatban bemutatott megoldás a felügyelt tanulás kategóriába sorolható, ugyanis általam készített képekből előállított, annotált adathalmazon tanítom a választott YOLO modellt.

2.4. Optikai Karakterfelismerés

Az optikai karakterfelismerés (OCR, Optical Character Recognition) olyan technológia, amely képes nyomtatott vagy kéziratos szöveget tartalmazó képekből automatikusan kinyerni az adatokat, és azt szerkeszthető, géppel olvasható formátummá alakítani. A technológia fejlődése az 1970-es évekhez vezethető vissza, amikor Ray Kurzweil egy olyan rendszer kifejlesztésén dolgozott, amely tetszőleges betűtípust képes felismerni [19]. Az általa alapított cég később egy olvasógépet hozott létre látássérültek számára, amely szöveget olvasott fel géphangon. Azóta az OCR jelentős fejlődésen ment keresztül, és ma már széles körben alkalmazzák, többek között mobiltelefonos dokumentumbeolvasásban, felhőszolgáltatásokban és automatizált rendszámfelismerésben is. Az OCR szoftverek felismerik az egyes karaktereket, majd szavakká, végül mondatokká rendezik azokat, így lehetővé téve a dokumentumok digitális feldolgozását, kereshetőségét és archiválását, jelentősen csökkentve a kézi adatbevitel szükségességét és hibalehetőségét. Működése az alábbi fő szakaszokra bontható:

1. Képrögzítés (Image Acquisition): Első lépésként a papíralapú dokumentumot digitalizálják (szkennerrel vagy kamerával) és a rendszer fekete-fehér (bináris) képpé alakítja azt. Ezután a kép világos és sötét részeit elemzi: a sötét területeket karakterként, a világos részeket háttérként azonosítja.
2. Előfeldolgozás (Preprocessing): A digitális kép megtisztítása történik, hogy javítsák a felismerés pontosságát. Ez magában foglalhatja a kép elforgatásának (deskewing) korrekcióját, a grafikai elemek (keretek, vonalak) eltávolítását, valamint annak megállapítását, hogy kézzel írt vagy nyomtatott szövegről van-e szó.

2. FEJEZET: ELMÉLETI HÁTTÉR

3. Oldalszerkezet-felismerés (Layout Recognition): Fejlettebb OCR rendszerek képesek nemcsak a karaktereket, hanem a teljes dokumentumszerkezetet is elemezni. Ez magában foglalja a szövegdobozok, táblázatok, képek elkülönítését, majd ezek sorokra, szavakra, végül karakterekre bontását.
4. Szövegfelismerés (Text Recognition): Ebben a szakaszban az OCR rendszer a sötét részeket karakterekként kezeli, és megpróbálja azonosítani azokat betűkként, számjegyekként vagy írásjelekként. Ez kétféle módszerrel történhet:
 - Mintafelismerés (Pattern Recognition): A rendszer korábban betanított karaktermintákat (ún. glifákat) használ az ismerős betűtípusok alapján történő felismeréshez. Ez a módszer akkor működik jól, ha a karakterek egy ismert betűtípushoz tartoznak [32],
 - Jellemzőalapú felismerés (Feature Recognition): Ha a rendszer ismeretlen betűtípust talál, akkor a karakterek szerkezetét (például vonalak, ívek, metszéspontok) elemzi. Például az "A" betű két átlós és egy vízszintes vonalból áll. E logikai szabályok alapján történik a karakterek azonosítása [16].

Miután egy karaktert sikeresen beazonosított, azt a rendszer gépi kódra, jellemzően ASCII-formátumra alakítja, így az a további feldolgozásra alkalmassá válik.

5. Utófeldolgozás (Postprocessing): A felismert szövegen nyelvi és szerkezeti korrekciók kerülnek alkalmazásra a pontosság javítása érdekében. Ez magában foglalhatja a szótár-alapú helyesírás-ellenőrzést, kontextualapú hibajavítást, vagy akár szerkezeti szabályok érvényesítését.

2.4.1. Az OCR technológiák fejlődése

Az optikai karakterfelismerés fejlődése az elmúlt évtizedekben szorosan követte a számítógépes látás és a mesterséges intelligencia fejlődését. Az OCR rendszerek több jelentős technológiai korszakon mentek keresztül.

- **Szabályalapú és mintafelismerő rendszerek:** A korai OCR rendszerek előre definiált karakterformák és kézzel megalkotott szabályok alapján működtek [19]. Ezek a módszerek elsősorban strukturált, jó minőségű dokumentumokon működtek hatékonyan.

2. FEJEZET: ELMÉLETI HÁTTÉR

- **Gépi tanulás alapú rendszerek:** Statisztikai modelleket és klasszikus gépi tanulási algoritmusokat alkalmaztak a karakterek felismerésére és osztályozására [8].
- **CNN-alapú modellek:** A konvolúciós neurális hálózatok (*CNN*) lehetővé tették az automatikus jellemzőkinyerést, jelentősen javítva a felismerési pontosságot [23].
- **RNN/LSTM alapú modellek:** A rekurzív neurális hálók (RNN) és az LSTM architektúrák alkalmassá váltak a szekvenciális szövegek feldolgozására. Ezeket gyakran CTC *Connectionist Temporal Classification* veszteségfüggvénnyel kombinálták [18, 17].
- **Transzformer-alapú modellek:** A transzformer architektúrák az önfigyelési mechanizmus (*self-attention*) segítségével képesek a globális szövegösszefüggések modellezésére, ami tovább javította az OCR rendszerek teljesítményét [41].

A fejlődés során több meghatározó OCR rendszer és modell is megjelent:

- **Tesseract OCR:** Az egyik legismertebb nyílt forráskódú OCR rendszer, amelyet eredetileg a Hewlett-Packard fejlesztett, majd később a Google továbbfejlesztett [30].
- **CRNN (Convolutional Recurrent Neural Network):** A CNN és RNN architektúrák kombinációjára épülő modell, amely hosszú ideig az egyik legelterjedtebb deep learning alapú OCR megoldásnak számított [23].
- **TrOCR (Transformer-based Optical Character Recognition):** A Microsoft által bemutatott modern OCR architektúra, amely transzformer encoder–decoder felépítést alkalmaz, és közvetlenül képekből generál szöveget [24].

2.4.2. Mi a TrOCR?

A **TrOCR**, azaz Transformer-based Optical Character Recognition, egy Microsoft által fejlesztett architektúra, ami két előre tanított transzformer modellt kombinál a képen lévő szöveg felismeréséhez [24].

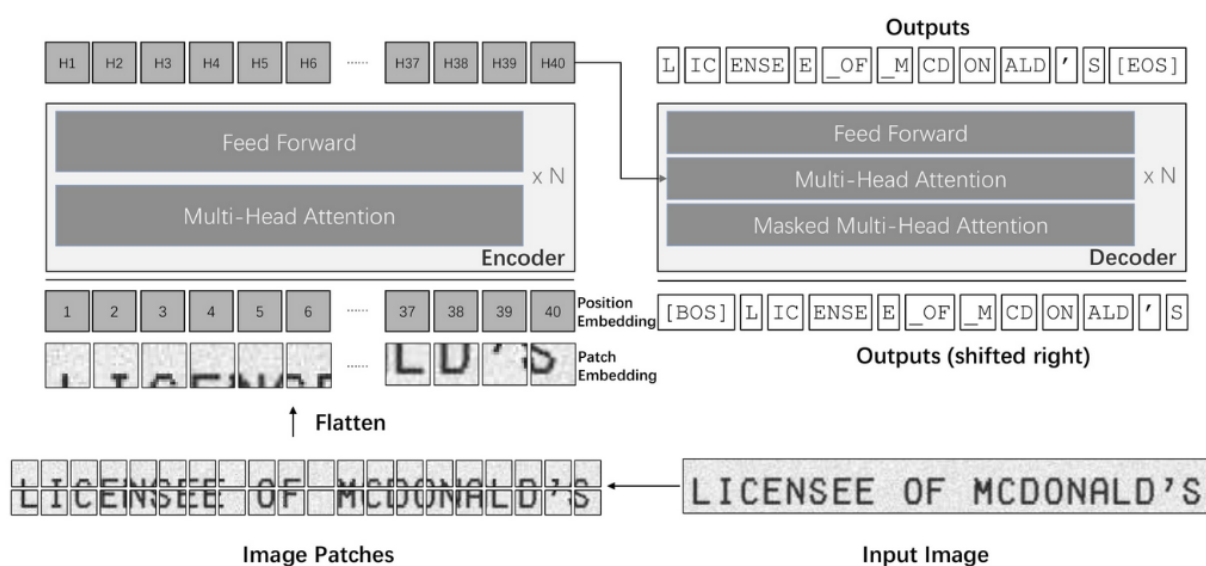
A TrOCR architektúrája

A TrOCR egy **encoder-decoder** felépítésű modell, amelynek 2.1. ábrán látható architektúrája két fő részből áll:

2. FEJEZET: ELMÉLETI HÁTTÉR

- **Kódoló (Encoder):** A képi encoder a **BEiT** ([6]) modell súlyaival van inicializálva. A bemeneti képet 16×16 pixeles patch-ekre osztja, amelyeket lineárisan leképez, majd abszolút pozíció-embeddingekkel egészít ki. Ezeket a sorozatokat dolgozzák fel a transformer encoder rétegei Multi-Head Attention és Feed Forward blokkok segítségével.
- **Dekódoló (Decoder):** A szöveges decoder a **RoBERTa** ([25]) modell súlyaival van inicializálva. Masked Multi-Head Attention és Multi-Head Attention rétegekből épül fel, és az encoder által kinyert vizuális jellemzők alapján autoregresszív módon generálja a kimeneti szöveget, minden új tokenet az eddig generált tokenek és a vizuális kontextus együttesen határoz meg.

Tanítás: A modell kétlépéses tanítási folyamaton esik át: először nagy mennyiségű szintetikus adaton történik az előtanítás, majd feladat-specifikus finomhangolás (fine-tuning) következik, például nyomtatott vagy kézírással szöveg felismerésére.



2.1. ábra. A TrOCR architektúrája [24].

2.4.3. Miért a TrOCR?

A TrOCR a karakterfelismerést **Seq2seq** feladatként kezeli: a modell a bemeneti képet egy vizuális jellemzősorozattá alakítja, majd ebből a sorozatból kimeneti szöveg-tokenek sorozatát generálja, anélkül, hogy hagyományos OCR folyamatokra (pl. CNN jellemzőkinyerés, CTC dekódolás) lenne szükség.

2.4.4. Metrikák

Az optikai karakterfelismerő (*OCR*) modellek értékelésére több különböző metrika használható. Ezek célja annak meghatározása, hogy a modell milyen pontossággal képes felismerni és rekonstruálni a képeken található szöveget.

A TrOCR modell értékelése során a legfontosabb metrikák közé tartozik a felismerési arány (*Recognition Rate*), a karakterhiba-arány (*Character Error Rate (CER)*) és a felidézési arány (*Recall Ratio*).

- **Felismerési Arány:** a helyesen felismert karakterek vagy teljes szövegek arányát méri az összes predikcióhoz viszonyítva. Magas pontosság esetén a modell a legtöbb karaktert vagy szót helyesen ismeri fel,
- **Karakterhiba-arány:** azt mutatja meg, hogy a predikált szöveg mennyiben tér el a való szövegtől karakter szinten. A karakterbeszúrásokat, törléseket és cseréket figyelembe véve kerül kiszámításra:

$$CER = \frac{S + D + I}{N}$$

ahol:

- * S a helyettesítések (*substitutions*) száma,
- * D a törlések (*deletions*) száma,
- * I a beszúrások (*insertions*) száma,
- * N pedig a szöveg karaktereinek száma.

A kisebb CER érték jobb teljesítményt jelent.

- **Felidézési Arány:** azt méri, hogy a modell a ténylegesen létező karakterek vagy szövegrészek mekkora részét ismerte fel sikeresen. Magas felidézési arány esetén a modell kevés releváns karaktert hagy figyelmen kívül.

2.5. Objektumdetektálás

Az objektumdetektálás (*object detection*) a számítógépes látás (*computer vision*) egyik alapvető feladata, célja objektumok felismerése és lokalizálása képeken vagy képkockákra darabolt videókon. A feladat két fő részből áll:

2. FEJEZET: ELMÉLETI HÁTTÉR

- az objektum kategorizálása,
- az objektum pozíciójának meghatározása, általában *bounding box* segítségével.

2.5.1. Klasszikus objektumdetektálás

Az objektumdetektálás az elmúlt évtizedekben jelentős változásokon ment keresztül. A mélytanulási (*deep learning*) modellek megjelenése előtti megközelítések hagyományos képfeldolgozási és gépi tanulási módszereken alapult, amelyek kézzel tervezett jellemzőkre (*hand-crafted features*) épültek, előre definiált szabályok és matematikai leírók segítségével próbálták felismerni az objektumokat a képeken.

A legismertebb klasszikus módszerek közé tartoznak [5]:

- **Haar Cascade Classifier:** Haar-szerű jellemzők (*Haar-like features*) és AdaBoost alapú osztályozás kombinációját alkalmazó módszer, amelyet elsősorban arcfelismerési feladatokra használtak.
- **HOG (Histogram of Oriented Gradients):** célja az objektumok alakját és körvonalát a képen található éirányok és struktúrák alapján reprezentálni [13].
- **SVM (Support Vector Machine):** klasszikus gépi tanulási osztályozó algoritmus, amely a különböző objektumosztályok közötti optimális elválasztó sík meghatározására törekszik [40].
- **SIFT (Scale-Invariant Feature Transform):** lokális jellemzőkinyerő módszer, amely robusztusan képes kulcspontokat detektálni méret-, forgatás- és megvilágításváltozások mellett is [26].
- **SURF (Speeded-Up Robust Features):** a SIFT gyorsabb alternatívája, amely közelítő számítások segítségével valós idejű alkalmazásokban is hatékonyabb működést biztosít [7].

Ezek a módszerek elsősorban egyszerűbb környezetekben működtek hatékonyan, azonban változó fényviszonyok, eltakarás vagy komplex háttér esetén teljesítményük jelentősen romlott.

2.5.2. Mélytanulás alapú objektumdetektálás

A konvolúciós neurális hálózatok (*Convolutional Neural Networks, CNN*) elterjedésével az objektumdetektálás pontossága jelentősen javult. A modern objektumdetektáló modellek két fő kategóriába sorolhatók:

Two-stage modellek

A two-stage modellek két lépésben végzik az objektumdetektálást:

1. először objektumjelölteket (*region proposals*) generálnak,
2. majd ezeket osztályozzák és finomítják.

A legismertebb two-stage modellek: R-CNN, Fast R-CNN, R-CNN; ezek a modellek a CNN-alapú objektumdetektálás első jelentős áttörései közé tartoznak. Az **R-CNN** vezette be a regionális objektumjelöltek és a mély neurális hálók kombinációját. A **Fast R-CNN** továbbfejlesztette ezt azáltal, hogy a konvolúciós számításokat megosztotta a teljes képen, így jelentősen gyorsabb tanítást tett lehetővé. A **Faster R-CNN** további optimalizálást vezetett be a *Region Proposal Network (RPN)* segítségével, amely már a hálózaton belül generálja a javasolt régiókat, ezáltal javítva a hatékonyságot és a pontosságot. Bár ezen modellek általánosságban nagy pontosságot érnek el, mivel számításigényük magasabb, ezért valós idejű felhasználásra kevésbé alkalmasak [28].

Single-stage modellek

A single-stage modellek az objektumok lokalizálását és osztályozását egyetlen neurális hálózati futtatással végzik el. Ez jelentősen gyorsabb működést tesz lehetővé.

A legismertebb single-stage modellek: YOLO (*You Only Look Once*), SSD (*Single Shot MultiBox Detector*), RetinaNet.

A **YOLO** megközelítés a bemeneti képet rácsokra (*grid*) osztja, és minden cellához közvetlenül jósl bounding boxokat és osztályvalószínűségeket. Ez a felépítés rendkívül gyors inferenciát tesz lehetővé, ezért a YOLO modellek különösen alkalmasak valós idejű alkalmazásokra [28].

Az **SSD** a detektálást több különböző mélységű feature map-en végzi, így képes különböző méretű objektumok hatékony felismerésére [9].

2. FEJEZET: ELMÉLETI HÁTTÉR

A **RetinaNet** egyik legfontosabb újítása a *Focal Loss* bevezetése, amely az osztályok ki-egyensúlyozatlanságát hivatott beállítani, ezáltal egyensúlyt teremtve sebesség és pontosság között [9].

Összességében a single-stage modellek célja a valós idejű működés, kellően magas detektálási pontosság biztosítása mellett, így széles körben alkalmazhatók gyakorlati rendszerekben.

2.5.3. Teljesítmény evaluálására szolgáló fő metrikák

Az objektumdetektáló modellek értékelése során több, egymást kiegészítő metrikát alkalmaznak, amelyek a pontosságot és a futási sebességet egyaránt jellemzik.

A leggyakrabban használt teljesítménymutatók közé tartozik a **mean Average Precision (mAP)**, az **Intersection over Union (IoU)**, valamint a **Frames Per Second (FPS)**. Ezek a mutatók együttesen adnak átfogó képet a modell hatékonyságáról.

- **IoU**: megadja, hogy a predikált és a valós bounding boxok milyen mértékben fedik egymást,
- **FPS**: a modell futási sebességét jellemzi, amely különösen fontos szempont valós idejű alkalmazások esetén,
- **mAP**: a detektálási pontosságot méri különböző objektumosztályok esetén, figyelembe véve a precision és recall értékeket.

2.5.4. Hatékonysági mutató - Mean Average Precision (mAP)

A **mean Average Precision (mAP)** az objektumfelismerő modelleknek, így a YOLO család tagjainak is egyik legfontosabb teljesítménymutatója. Egyetlen, összegző értékkel méri, mennyire pontosan és megbízhatóan azonosítja és lokalizálja a modell a különböző objektumosztályokat. [34]

Alapfogalmak:

- **Precízió (Precision)**: A helyes találatok aránya az összes találathoz képest. Magas precízió esetén kevés a hamis pozitív eredmény.
- **Recall**: A helyes találatok aránya az összes valódi objektumhoz képest. Magas recall esetén a modell megtalálja a legtöbb releváns objektumot.

2. FEJEZET: ELMÉLETI HÁTTÉR

E két mutató gyakran ellentétes irányba hat, a mAP ezeket egyensúlyba hozva ad átfogó képet a modell teljesítményéről.

A mAP kiszámításának lépései:

1. **Konfidencia szerinti rendezés:** A modell által detektált objektumokat biztonság (confidence score) alapján sorba rendezzük.
2. **Precision–Recall görbe:** Minden objektumosztályra külön-külön ábrázoljuk a precízió és recall változását a küszöbök mentén.
3. **AP (Average Precision):** A precision–recall görbe alatti terület, egy adott osztályra.
4. **mAP:** Az összes osztály AP értékének átlaga.

Gyakori változatai:

- **mAP@0.5:** Az objektum akkor számít helyesen detektáltnak, ha az átfedés (IoU) a földi igazsággal legalább 0,5.
- **mAP@.5:.95:** Az AP értékeket több IoU küszöbön átlagoljuk (0,5–0,95, 0,05-ös lépésekkel). Ez részletesebb és szigorúbb értékelést ad.

Miért fontos a mAP? A mAP egyszerre méri az osztályozási és lokalizációs pontosságot, így komplex feladatoknál – például autonóm járművek, egészségügyi képfeldolgozás – megbízható értékelési mutató. A mAP javítása érdekében gyakori technikák:

- minőségi annotáció,
- adatbővítés (data augmentation),
- megfelelő modellarchitektúra kiválasztása (pl. YOLOv11),
- hiperparaméterek finomhangolása.

2.5.5. Mi az a YOLO?

A **YOLO** (You Only Look Once) egy népszerű, nyílt forráskódú [21] objektumfelismerő és képszegmentáló *out-of-the-box* mélytanulási modell, amelyet *Joseph Redmon* és *Ali Farhadi* fejlesztett ki a Washingtoni Egyetemen. A modell első változata 2015-ben jelent meg, és

gyorsaságával, valamint pontosságával rövid időn belül nagy népszerűsége tett szert. YOLO különlegessége abban rejlik, hogy az objektumfelismerést egyetlen neurális hálózati futtatással végzi el, szemben más módszerekkel, amelyek több lépésben dolgoznak. Ezáltal valós idejű felismerésre is kiválóan alkalmas [39].

A 2024-ben megjelent, az *Ultralytics* csapata által fejlesztett **YOLOv11** verziót használtam ezen szakdolgozat során.

2.5.6. Miért YOLOv11?

A YOLOv11 jelenleg az egyik legmodernebb, *state-of-the-art* teljesítményt nyújtó modell az objektumfelismerés területén. Nem csupán objektumdetekcióra, hanem képszegmentálásra, nyomkövetésre és képosztályozásra is kiválóan alkalmazható. Az ilyen típusú modellek rendkívül sokoldalúak, és hatékonyan bevethetők a mesterséges intelligencia számos alkalmazási területén, a közlekedéstől az ipari automatizáláson át a biztonságtechnikai rendszerekig [21].

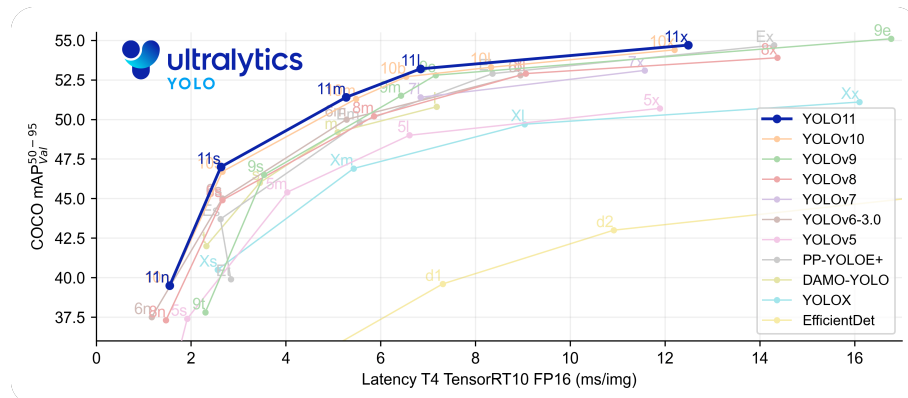
2.5.7. A YOLOv11 főbb jellemzői

A YOLOv11 modell a számítógépes látás egyik legkorszerűbb megoldását képviseli, számos fejlesztést tartalmazva az előző verziókhoz képest [37]. Az alábbiakban összefoglalom legfontosabb tulajdonságait:

- **Fokozott jellemzőkinyerés:** A YOLOv11 továbbfejlesztett *backbone* és *neck* architektúrával rendelkezik, amely lehetővé teszi a képi jellemzők pontosabb és mélyebb kinyerését. Ez pontosabb objektumfelismerést biztosít.
- **Hatékonyagra és sebességre optimalizálva:** A modell finomhangolt architektúrát és optimalizált tanítási folyamatot alkalmaz, melynek eredménye a gyorsabb képfeldolgozás, miközben az észlelési pontosság és a teljesítmény közötti egyensúlyt is megőrzi.
- **Nagyobb pontosság kevesebb paraméterrel:** A YOLOv11m variáns 22%-kal kevesebb paramétert használ, mint a YOLOv8m, mégis magasabb **mAP** (mean Average Precision) értéket ér el a COCO adathalmazon. Ez azt jelenti, hogy a modell számítási szempontból hatékonyabb, miközben nem csökken a pontosság.
- **Adaptálhatóság:** A YOLOv11 könnyedén telepíthető különféle rendszerekre, beleértve a felhőalapú platformokat és NVIDIA GPU-val rendelkező rendszereket, ezáltal maximális rugalmasságot biztosít.

2. FEJEZET: ELMÉLETI HÁTTÉR

- **Széleskörű feladat-támogatás (Broad Range of Supported Tasks):** A modell alkalmas objektumdetektálásra, szegmentálásra (instance segmentation), képosztályozásra, testtartásbecslésre (pose estimation) és orientált objektumfelismerésre (OBB) is, így változatos számítógépes látási problémákra nyújt hatékony megoldást.



2.2. ábra. YOLO verziók összehasonlítása.

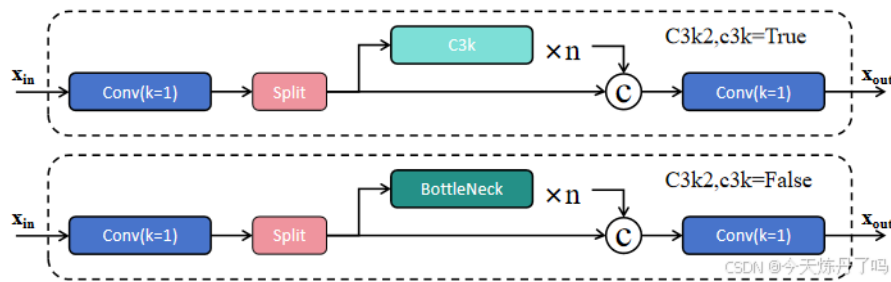
2.5.8. A YOLOv11 architektúrája és rétegei

A YOLOv11 modell egy **egylépcsős (single-stage)** objektumdetektáló architektúra, amely három fő komponensből épül fel: *backbone*, *neck* és *head*. Ezek a komponensek együtt biztosítják a hatékony jellemzőkinyerést és a pontos objektumfelismerést [22].

Backbone

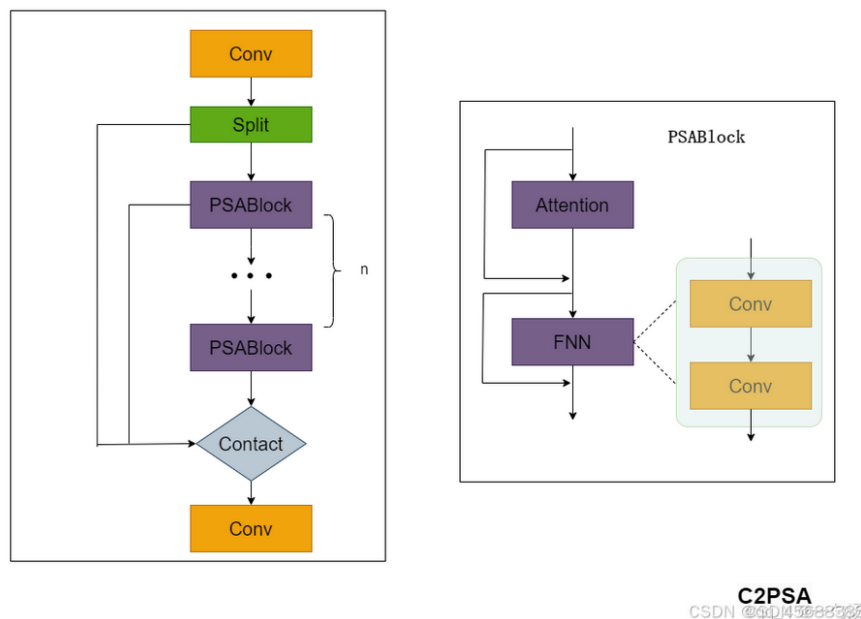
A backbone feladata a bemeneti kép jellemzőinek több skálán történő kinyerése. Ezt a YOLOv11 úgy végzi el, hogy megtart egy, az elődei által használt struktúrát, tehát konvolúciós rétegeket (convolutional layers) használ a bemeneti kép lebontására. A YOLOv11 egyik fontos fejlesztése a **C3k2** (2.3 ábra) blokk bevezetése, amely a YOLOv8 modellben alkalmazott **C2f** blokkot váltotta. A C3k2 blokk növeli a jellemzőkinyerés hatékonyságát, úgy, hogy közben csökkenti a számítási költséget, ugyanis egy nagy konvolúció helyett két kisebbet használ [15].

2. FEJEZET: ELMÉLETI HÁTTÉR



2.3. ábra. A C3k2 blokk felépítése. Forrás: [31].

Jelentős újdonság az is, hogy a korábbi verziók SPPF (Spatial Pyramid Pooling - Fast) blokkjának megtartása mellett, a YOLOv11 bevezet egy **C2PSA** (Cross Stage Partial with Spatial Attention) blokkot (lásd 2.4. ábra), ami javítja a feature map-ek 'térbeli figyelmét', ezáltal lehetővé teszi, hogy a modell jobban fókuszáljon fontosabb területekre egy adott képen, potenciálisan növelve a detekciós pontosságot [22].



2.4. ábra. A C2PSA blokk felépítése. Forrás: [29].

Neck

A neck rész a különböző méretű feature map-ek egyesítéséért felelős, és az így előállított jellemzőket továbbítja a head rész felé a végső predikciókhoz. Ez a folyamat tipikusan upsampling és különböző szintekről származó jellemzők összefűzésével valósul meg, lehetővé téve a többskálás információ hatékony feldolgozását.

2. FEJEZET: ELMÉLETI HÁTTÉR

A *backbone*-hoz hasonlóan, korábban alkalmazott C2f blokkokat a *neck* részben is **C3k2 blokkok** váltják fel. Ezek a blokkok az upsampling és concatenation lépések után kerülnek alkalmazásra, és céljuk a jellemzők hatékonyabb feldolgozása.

Head

A head réteg végzi az objektumok tényleges felismerését és lokalizálását. A neck által előállított jellemzőtérképeket feldolgozva a modell meghatározza az objektumok helyét, kategóriáját és a detekció megbízhatóságát.

- **Bounding box regresszió:** meghatározza az objektumok helyét és méretét,
- **Osztályozás:** megállapítja az objektum kategóriáját,
- **Confidence score:** a detekció megbízhatóságát jelzi.

A YOLOv11 a head részben is alkalmaz C3k2 blokkokat, amelyek a feature map-ek további finomítását és hatékonyabb feldolgozását segítik különböző skálákon. Ezek a blokkok hozzájárulnak a modell pontosságának növeléséhez és a számítási hatékonyság javításához.

A YOLOv11 **anchor-free** megközelítést alkalmaz, amely egyszerűsíti a tanítási folyamatot, és csökkenti a kézzel definiált anchor boxok szükségességét, ezáltal növelve az általános rugalmasságot és teljesítményt.

2.5.9. A YOLOv11 által támogatott fő számítógépes látási feladatok

A YOLOv11 modell sokoldalúságát jól mutatja, hogy számos különböző számítógépes látási (*computer vision*) feladat megoldására alkalmas [38]. Az alábbiakban a legfontosabb alkalmazási területeket ismertetem:

- **Objektumdetektálás (Object Detection):** A modell képes objektumok azonosítására és lokalizálására képeken vagy videókon, *bounding box* koordináták segítségével. Ez alapvető szerepet játszik például autonóm járművekben, megfigyelőrendszerekben és kiskereskedelmi analitikában.
- **Szegmentálás (Instance Segmentation):** Az objektumok nemcsak felismerésre kerülnek, hanem pixelpontossággal elkülöníthetők egymástól. Ez különösen fontos orvosi képfeldolgozásban vagy ipari hibadetektálás során.

2. FEJEZET: ELMÉLETI HÁTTÉR

- **Képosztályozás (Image Classification):** A YOLOv11 teljes képeket képes kategóriákba sorolni. Tipikus alkalmazási területe az e-kereskedelem (pl. termékosztályozás) vagy ökológiai megfigyelések.
- **Testtartásbecslés (Pose Estimation):** A modell kulcsponthozokat (*keypoints*) detektál az emberi testen vagy más objektumokon, lehetővé téve a mozgás és testtartás elemzését. Ez hasznos sportanalitikában, egészségügyben és fitness alkalmazásokban.
- **Orientált objektumdetektálás (Oriented Object Detection):** A YOLOv11 képes forgatott objektumok detektálására is, nem csak tengelyekkel párhuzamos határoló dobozokkal. Ez különösen fontos légi felvételek elemzésénél, robotikában és raktárazomatizálásban.
- **Objektumkövetés (Object Tracking):** A modell képes objektumok mozgásának követésére egymást követő képkockákon keresztül. Ez kulcsfontosságú például forgalomelemzésben, sportesemények feldolgozásában és biztonsági rendszerekben.

3. fejezet

Automatikus rendszámleolvasás Raspberry Pi-n

Összefoglaló: Ez a fejezet a projekt megvalósítását mutatja be a választott technológiai eszközöktől a teljes pipeline működéséig. Ismerteti a Python programozási nyelv alkalmazásának indokait, bemutatja az adathalmaz előállításának folyamatát, valamint a Raspberry Pi 5-öt, mint kitelepítési céleszközt. Ezt követően részletezi a YOLOv11 modell tanítási eredményeit offline-, illetve online augmentálás mellett, majd összehasonlítja a tesztelt karakterfelismerési modellek teljesítményét általános és szélsőséges pozícióban lévő rendszámok esetén egyaránt. Végül lépésről lépésre bemutatja a teljes rendszer működési folyamatát, rendszámdetektálástól egészen az utófeldolgozott eredmények fájlba való logolásáig.

3.1. Használt programozási nyelv - Python

3.1.1. Miért ideális nyelv a Python mesterséges intelligencia projektekhez?

A Python nyelv szinte ipari szabvánnyá vált a mesterséges intelligencia, gépi tanulás, mélytanulás és számítógépes látás területein. Ennek fő okai a következők:

Széleskörű könyvtártámogatás

Számos nyílt forráskódú könyvtár áll rendelkezésre, például:

- NumPy [1], Pandas – adatkezelés és feldolgozás
- PyTorch [2] – mélytanulási modellek fejlesztéséhez
- OpenCV [3], YOLO – számítógépes látás alkalmazásokhoz

Erős közösségi háttér

A Python fejlesztői közössége hatalmas és aktív. Ennek köszönhetően rengeteg fórum és oktatóanyag áll rendelkezésre, a könyvtárak jól dokumentáltak és gyakran frissülnek.

3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

Gyors prototípus-készítés

A Python szintaxisa lehetővé teszi, hogy a fejlesztők gyorsan kipróbáljanak ötleteket, így különösen alkalmas kutatási célokra és kísérleti MI rendszerek fejlesztésére.

Integrálhatóság

A Python könnyen összekapcsolható más rendszerekkel és nyelvekkel:

- REST API-k és webes alkalmazások (pl. Flask, Django),
- fizikai eszközök vezérlése (pl. Raspberry Pi),
- C/C++ vagy Java modulokkal való együttműködés.

Tehát a Python egy olyan eszköz, amely ötvözi az egyszerűséget, a hatékonyságot és a fejlett könyvtári ökoszisztémát. Ennek köszönhetően a mesterséges intelligencia projektek legjobb választása kutatóknak és fejlesztőknek egyaránt.

3.2. Adathalmaz

Ahogy az elméleti áttekintőben már említettem, a kiválasztott modellt, ami jelen esetben az *Ultralytics* csapata által fejlesztett YOLOv11, egy adathalmazon tanítom. Az adathalmazt a következő lépések mentén állítottuk elő:

1. Képek készítése

Az adathalmaz képei manuálisan, iPhone 15-tel készültek, kizárólag természetes, többnyire nappali fényviszonyok mellett, városi és falusi közlekedési környezetben. Mozgó és álló járművekről egyaránt készültek képek, különböző távolságokból és szögekből, valós forgalmi helyzetekben. Az adathalmaz sokszínűségét növeli, hogy a képek eltérő időpontokban, különböző helyszíneken készültek, így megvilágítási- és háttérkörülmények szempontjából is változatos.

2. Képek címkézése Roboflow segítségével

A számítógépes látás feladatok, ebben a konkrét esetben a rendszámfelismerés hatékony megoldásához elengedhetetlen egy jól strukturált és pontosan annotált adathalmaz. A kézi annotálás időigényes folyamatát jelentősen megkönnyítik olyan online platformok, mint a Roboflow, amely egy könnyen kezelhető, böngészőalapú annotáló- és adatkezelő rendszer.

A Roboflow annotálási folyamata a következő lépésekből áll:

- (a) **Projekt létrehozása:** A felhasználó regisztráció után új projektet hozhat létre, ahol megadhatja a projekt típusát (pl. *Object Detection*) és a címkék listáját (ebben a projektben 2 féle címkét használunk: *carplate*, *red_carplate*).
- (b) **Képek feltöltése:** A rendszer lehetőséget biztosít képek vagy mappák közvetlen feltöltésére. Többféle formátumot (JPEG, PNG stb.) és akár ZIP fájlokat is támogat.
- (c) **Annotálás:** A Roboflow beépített annotáló eszközeivel a képeken egyszerűen, egérrel rajzolhatók be a kívánt objektumok köré a keretek. Két annotálási eszközt/módot alkalmaztam:
 - **Bounding Box:** egyszerű tengelyekkel párhuzamos téglalap alakú keret az objektum köré,
 - **Polygon :** a rendszám tényleges szögéhez igazított, elforgatott téglalap alakú keret, amely pontosabban illeszkedik a ferde vagy döntött rendszámokra.

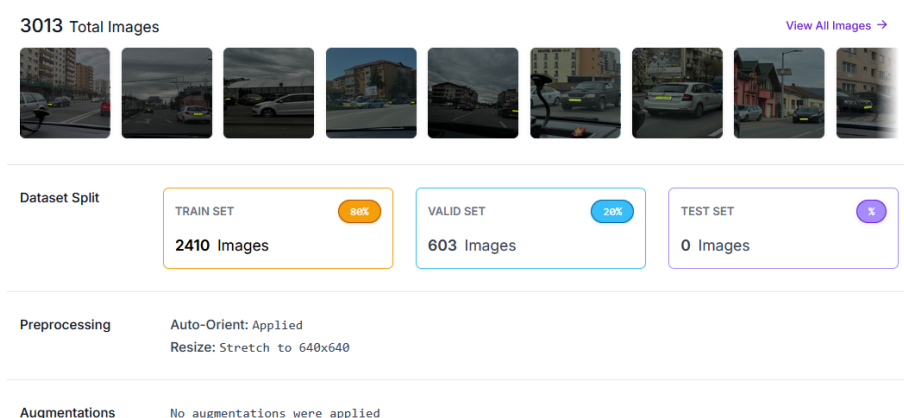
3. Exportálás

Az annotált adatok különböző formátumokban exportálhatók (YOLO, COCO, Pascal VOC stb.), így könnyen illeszthetők különböző gépi tanulási modellekhez. Kezdetben a sima **YOLOv8** formátumot használtuk, aminek kiválasztása esetén a rendszer minden képhez generál egy `.txt` fájlt, ami tartalmazza a képen szereplő objektumok osztályozósíkját, illetve a tengelyekkel párhuzamos keret (bounding box) középpontját és méreteit (`class_id x_center y_center width height`). Később áttértem a **YOLOv8-OB** formátumra, amely szintén egy `.txt` fájlt generál minden képhez, azonban az elforgatott keret (*oriented bounding box*) négy sarkának koordinátáit tartalmazza (`class_id x1 y1 x2 y2 x3 y3 x4 y4`). Ezt az opciót használtuk az adathalmaz végső verziójának exportálásakor.

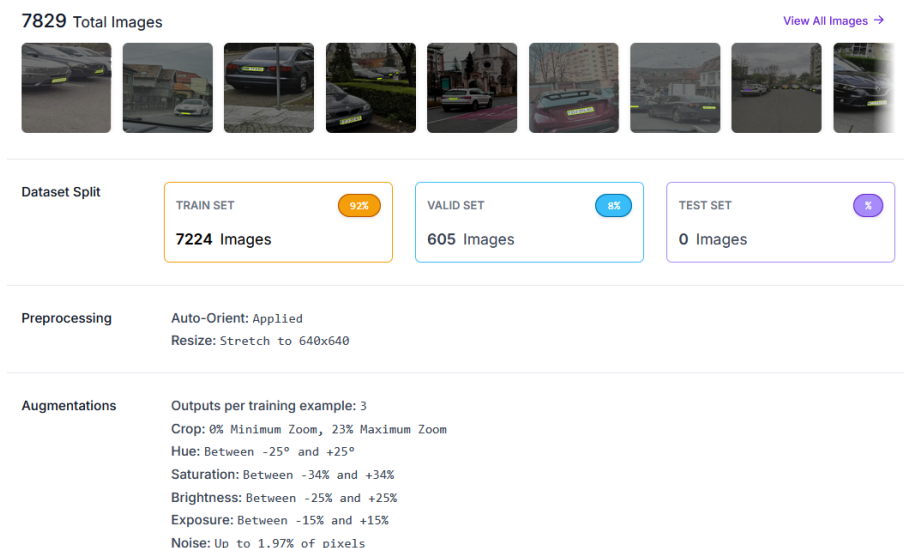
3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

A 3.1. ábrán az offline augmentációk nélküli adathalmaz látható a Roboflow felületén, a 3013 képből álló adathalmazt szemlélteti, 80-20%-os tanítási-validációs felosztással.

A 3.2. ábra ezzel szemben egy sokkal nagyobb, 7829 képből álló adathalmazt prezentál, amely a kezdeti 3013 képből kiindulva, offline augmentációk alkalmazásának eredményeként duzzadt ekkorára. Képenként 3 új verzió készül, ez eredményezi az ábrán látható 92-8%-os tanító-validációs felosztást.



3.1. ábra. Roboflow felülete offline augmentációk alkalmazása nélkül.



3.2. ábra. Roboflow felülete offline augmentációk alkalmazása esetén.

Roboflow előnye, hogy egyetlen platformon kínál:

- egyszerű grafikus annotálási felületet,
- augmentálási lehetőségeket (pl. elforgatás, kontrasztnövelés),
- automatikus verziókezelést,
- közvetlen exportálást különböző gépi tanulási keretrendszerekhez.

3.3. Használt eszköz: Raspberry Pi 5

A Raspberry Pi 5 a Raspberry Pi Foundation által 2023-ban kiadott egykártyás számítógép (*single-board computer*), amely jelentős teljesítménybeli ugrást jelent elődjéhez, a Raspberry Pi 4-hez képest. A legfontosabb hardveres jellemzői:

- **Processzor:** Broadcom BCM2712, négymagos ARM Cortex-A76, 2.4 GHz
- **Memória:** 4 GB / 8 GB LPDDR4X RAM
- **Grafikus egység:** VideoCore VII GPU
- **Csatlakozók:** 2× USB 3.0, 2× USB 2.0, 2× micro HDMI, GPIO, CSI/DSI
- **Hálózat:** Gigabit Ethernet, WiFi 5, Bluetooth 5.0

3.3.1. Miért Raspberry Pi 5?

A Raspberry Pi 5 választása több szempontból is indokolt volt. Egyrészt a projekt célja egy valós környezetben telepíthető, kompakt rendszer kialakítása volt, amely nem igényel dedikált GPU-t vagy nagyteljesítményű asztali számítógépet. Másrészt a Pi 5 az előző generációhoz képest közel **kétszeres CPU teljesítményt** nyújt, ami elengedhetetlen közel valós idejű inferencia futtatásához. Az NCNN ([36]) formátumra exportált *yolov11n-obb* modell kifejezetten ARM alapú, erőforráskorlátozott eszközökre van optimalizálva, így a Raspberry Pi 5 ideális célplatformnak bizonyult. Továbbá a platform alacsony energiafogyasztása, kompakt mérete, ezáltal hordozhatósága, autóból viszonylag könnyedén üzemeltethetősége, és széles körű közösségi támogatása hosszú távon is fenntartható megoldást kínál.

3.4. Használt mesterséges intelligencia modell: YOLOv11

Ahogy korábban ismertettük, a rendszámok detektálására a YOLO modellcsalád egyik legújabb tagját, a YOLOv11-et alkalmazzuk. A következőkben ismertetjük a modell gyakorlati használatát, illetve bemutatásra kerülnek a különböző modellvariánsok tanítási eredményei mind offline, mind online augmentáció alkalmazása mellett, különböző hiperparaméter-konfigurációk esetén.

3.4.1. YOLOv11 használata

Az előre betanított (pre-trained) modell tanításához csupán pár egyszerű lépést kell megtennünk:

1. **ultralytics** Python csomag telepítése *pip*-el, a Python csomagkezelőjével.
2. A kiválasztott modell betöltése, tanítása:

```
from ultralytics import YOLO

# Előre betanított YOLOv11 modell betöltése (n, s, m, l vagy x verziók közül
↪ választhatunk)
model = YOLO("yolo11n.pt") # n - nano verzió
5
# Tanítás az általunk létrehozott/használt adathalmazon
model.train(data="path/to/dataset.yaml", epochs=100, imgsz=640)
```

3. A tanított modell exportálása:

```
from ultralytics import YOLO

# Modell betöltése (előre tanított vagy saját)
model = YOLO("yolo11n.pt")
5
# Exportáljuk a modellt NCNN formátumba
model.export(format="ncnn")
```

4. Inferencia futtatása, detekciók megjelenítése:

```
from ultralytics import YOLO
import cv2

# Exportált modell betöltése
5 ncnn_model = YOLO("yolo11n_ncnn_model")

# Inferencia képen
results = ncnn_model("image.jpg", verbose=False)
```

3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

```
10 for result in results:
    if result.obb is None:
        continue
    for obb in result.obb:
        points = obb.xyxyxyxy[0].cpu().numpy().astype(np.float32)
15     cv2.polylines(frame, [points.astype(np.int32).reshape((-1, 1, 2))],
        ↪ True, (0, 255, 0), 1)

    cv2.imshow("Detections", frame)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

20

# Inferencia kamerán
cap = cv2.VideoCapture(0)

25 while True:
    ret, frame = cap.read()
    if not ret:
        print("Could not read frame; exiting.")
        break

30     results = ncnn_model(frame, verbose=False)

    for result in results:
        if result.obb is None:
35             continue

        for obb in result.obb:
            points = obb.xyxyxyxy[0].cpu().numpy().astype(np.float32)
            cv2.polylines(frame, [points.astype(np.int32).reshape((-1, 1, 2))],
            ↪ True, (0, 255, 0), 1)

40     cv2.imshow("Detections", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

45 cap.release()
cv2.destroyAllWindows()
```

3.4.2. YOLOv11 modellek összehasonlítása

A modell kiválasztásánál két fő szempontot vettünk figyelembe: a detektálási pontosságot és a futási sebességet. A 3.1. és 3.2. táblázat a YOLOv11 model család különböző méretű változatainak teljesítményét hasonlítja össze a COCO [33] adathalmazon, 640 pixeles képekkel mérve. Megfigyelhető, hogy a nagyobb modellek ugyan magasabb mAP értéket érnek el, azonban CPU-n futtatva jelentősen lassabbak, a **YOLO11x** esetében ez közel **462 ms**-t jelent képkockánként, szemben a **YOLO11n** **56 ms**-es futási idejével. Az OBB változatok esetén hasonló tendencia figyelhető meg. Mivel a cél egy közel valós idejű, beágyazott rendszer kialakítása

3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

volt, ahol a sebesség kritikus tényező, a legkisebb, ugyanakkor kellően pontos **YOLO11n-obb** változat alkalmazása mellett döntöttünk, amely mindössze **2.7 millió** paraméterrel rendelkezik és CPU-n átlagosan **117.6 ms** alatt dolgoz fel egy képkockát.

Model	mAP _{val} 50–95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

3.1. táblázat. A YOLO11 modellek összehasonlítása különböző szempontok szerint [35].

Model	size (pixels)	mAP _{test} 50	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n-obb	1024	78.4	117.6 ± 0.8	4.4 ± 0.0	2.7	16.8
YOLO11s-obb	1024	79.5	219.4 ± 4.0	5.1 ± 0.0	9.7	57.1
YOLO11m-obb	1024	80.9	562.8 ± 2.9	10.1 ± 0.4	20.9	182.8
YOLO11l-obb	1024	81.0	712.5 ± 5.0	13.5 ± 0.6	26.1	231.2
YOLO11x-obb	1024	81.3	1408.6 ± 7.7	28.6 ± 1.0	58.8	519.1

3.2. táblázat. A YOLO11-OBb modellek összehasonlítása különböző szempontok szerint [35].

3.4.3. YOLO11-el elért tanítási eredmények

Ahogy azt korábban ismertettük, a kiválasztott YOLO modellt, a *yolo11n* változatot, valamint annak orientált határolókeretekkel dolgozó verzióját, a *yolo11n-obb* modellt, teljes egészében saját készítésű és saját annotációval ellátott képekből álló adathalmazon tanítottuk. A korábban bemutatott általános tanítási szkriptet a feladat sajátosságainak megfelelően módosítottuk, majd különböző offline augmentációkkal egészítettük ki, amelynek eredményeként jött létre a végleges tanítási szkript:

```
import torch
import os
from ultralytics import YOLO

5 def main():
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    print(f"Using device: {device}")
    if device == 'cuda':
        print(torch.cuda.get_device_name(0))
10
    model = YOLO('yolo11n-obb.pt').to(device)

    os.makedirs('runs', exist_ok=True)
```

3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

```
15 model.train(  
    data="datasets/CarplateDetection-24/data.yaml",  
    epochs=200,  
    batch=32,  
    imgsz=640,  
  
    hsv_h=0.015,  
    hsv_s=0.5,  
    hsv_v=0.3,  
  
    degrees=3,  
    translate=0.08,  
    scale=0.4,  
    shear=1.5,  
    perspective=0.0003,  
  
   fliplr=0.5,  
    flipud=0.0,  
  
    mosaic=0.5,  
    close_mosaic=10,  
    mixup=0.05,  
  
    amp=True,  
    patience=15,  
  
    lr0=0.001,  
    cos_lr=False,  
  
    project="runs",  
    name="train_results_v24_yolov11n-obb_cosFalse_lr001_b32_noroboaugm",  
45 )
```

Az első tizenegy tanítás egy NVIDIA RTX 3060 GPU-val rendelkező laptopon történt, ez csak minimális batch méretet tett lehetővé, köszönhetően a limitált, mindössze 6GB-os VRAM-kapacitásának. Az összes következő tanítás egy A100-as, 20GB VRAM-mal rendelkező kártyán történt, amely az egy tanítási epoch-hoz szükséges időt töredékére, 7 percről körülbelül 1 percre csökkentette, emellett jelentősen nagyobb batch méret használatát is lehetővé tette (mindkettő függött a kártya aktuális használati mértékétől).

A 3.3 táblázatban nyomon követhetőek az adathalmaz méretbeli növekedése mellett történő, különböző paraméterekkel való kísérletezéssel elért tanítási eredmények. Fontos megjegyezni, hogy ezen tanítások esetében az adathalmaz augmentálása **offline** módon, a Roboflow felületén történt, ami azt jelenti, hogy az augmentált képek előre legenerálásra kerültek és a tanítás során statikus adathalmazként lettek felhasználva.

3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

#	Model	Total Images	Epochs	Batch	LR0	CosLR	Best Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
1	yolo11n	887	250	8	0.003	False	123	0.9336	0.6772	0.7600	0.5056
2	yolo11n	1032	150	32	0.003	False	116	0.9390	0.7439	0.8372	0.5349
3	yolo11n	1032	150	32	0.005	False	88	0.9466	0.8015	0.8684	0.5590
4	yolo11n	887	200	8	0.010	False	149	0.9365	0.8456	0.8881	0.6076
5	yolo11n	2303	250	8	0.003	False	230	0.9454	0.8495	0.8870	0.6241
6	yolo11l	4751	100	4	0.010	False	53	0.8604	0.7255	0.8454	0.6314
7	yolo11n	1032	200	4	0.010	False	83	0.8849	0.7855	0.8360	0.6752
8	yolo11n	4751	250	8	0.005	False	158	0.8887	0.8918	0.9116	0.6955
9	yolo11n	7569	125	8	0.002	True	82	0.8624	0.8166	0.8652	0.7022
10	yolo11n	4751	200	4	0.010	False	102	0.9259	0.7931	0.8544	0.7067
11	yolo11n	7569	125	8	0.005	False	84	0.8800	0.7853	0.8632	0.7162
12	yolo11n-obb	7569	200	32	0.001	True	52	0.9252	0.8531	0.9125	0.8233
13	yolo11n-obb	7829	200	64	0.002	False	31	0.9496	0.8768	0.9420	0.8508
14	yolo11n-obb	7829	200	32	0.001	True	55	0.9777	0.8593	0.9329	0.8543
15	yolo11n-obb	7829	200	32	0.001	False	64	0.9761	0.8763	0.9224	0.8611

3.3. táblázat. Tanítási Eredmények Offline Augmentációt használva

#	Model	Total Images	Epochs	Batch	LR0	CosLR	Best Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
1	yolo11n-obb	3013	200	128	0.001	False	52	0.9871	0.8296	0.9198	0.8111
2	yolo11n-obb	3013	200	128	0.001	True	70	0.9696	0.8519	0.9277	0.8481
3	yolo11n-obb	3013	200	128	0.001	True	85	0.98	0.8984	0.9641	0.8741
4	yolo11n-obb	3013	200	32	0.005	True	70	0.9206	0.9251	0.9669	0.8802
5	yolo11n-obb	3013	200	128	0.001	True	132	0.9808	0.9259	0.9631	0.892

3.4. táblázat. Tanítási Eredmények Online Augmentációt Használva

Az **offline** augmentálással szemben lehetőség van **online** augmentálásra is, ebben az esetben az átalakítások valós időben, minden egyes tanítási lépés során véletlenszerűen kerülnek alkalmazásra, így a modell minden egyes epoch-ban eltérő változatát látja ugyanannak a képnek, ami hatékonyabb általánosítást eredményez.

Amint a 3.3. és a 3.4. táblázat mutatja, a végső offline augmentált adathalmazon elért legjobb **mAP@0.5:0.95** érték **0.8611** volt. Ezt sikerült felülmúlni online augmentálás alkalmazásával, amelynek eredményeként a végső, legjobb **mAP@0.5:0.95** érték **0.892**-re nőtt.

3.5. Használt karakterfelismerő modell: TrOCR

Ahogy korábban említettük, a projekt karakterfelismerő komponenseként a Microsoft által fejlesztett **TrOCR** modellcsaládot alkalmaztuk. A következőkben bemutatásra kerül a konkrét verzió kiválasztásának folyamata, a különböző verziók összehasonlítása, valamint a Raspberry Pi 5-ön mért teljesítménye.

Images	Model	Annotation	Recognition rate (%)	CER	Recall (%)
100	TrOCR-base	non-obb	63.00	0.1834	79.51
100	TrOCR-base	obb	79.00	0.0695	91.17
100	TrOCR-small	non-obb	39.00	0.3983	54.01
100	TrOCR-small	obb	75	0.1302	84.95
100	Tesseract	non-obb	15.00	0.6418	26.93
100	Tesseract	obb	42.00	0.3256	56.01
100	EasyOCR	non-obb	15.00	0.4628	40.26
100	EasyOCR	obb	30.00	0.3271	56.01

3.5. táblázat. Címke alapú kivágás eredményei

3.5.1. Melyik TrOCR verzió?

Kiindulópontként a **trocr-base-printed** modell került kiválasztásra, amelyet több alternatív OCR modellel, valamint annak kisebb, kevésbé robusztus változatával hasonlítottunk össze. Az összehasonlítás három standard, az OCR-rendszerek értékelésében gyakran alkalmazott metrika, nevezetesen a Recognition Rate, a Character Error Rate és Recall Ratio alapján történt. Az értékeléshez használt adathalmaz a YOLO detektálási lépéstől független módon került összeállításra: a rendszámképek kizárólag az annotált adathalmazból, az alap- illetve orientált határolókeretes formátumú címkék (*ground truth*) alapján kerültek kivágásra. Az így kapott képek nem részei a YOLO modell tanítására felhasznált adathalmaznak, ezáltal biztosítva az értékelés függetlenségét a detektálási lépéstől. Fontos megjegyezni, hogy az összehasonlítás során minden modell esetén egységes utófeldolgozás (*post-processing*) került alkalmazásra, amelynek célja a romániai rendszámformátum formai követelményeinek megfelelő karakterláncok érvényesítése volt.

Az 3.5. táblázat az általános adathalmazon mért eredmények alapján megállapítható, hogy a **TrOCR-base** modell OBB kivágások esetén érte el a legjobb felismerési arányt (**79%**), amelyet **0.0695**-ös CER érték és **91.17%**-os Recall kísér. A CER értéke azt jelzi, hogy a predikált karakterláncok átlagosan **6.95%**-ban térnek el a valós rendszámértékektől, míg a Recall érték azt mutatja, hogy a valóban létező karakterek közel **91%**-át sikeresen felismerte a modell. Ez jelentősen felülmúlja mind a Tesseract, mind az EasyOCR teljesítményét, sőt még a kisebb verziójának, a **TrOCR-small**-nak számait is minimálisan meghaladja. Szintén egyértelműen megfigyelhető, hogy az OBB alapú kivágások minden modell esetén jobb eredményt hoztak a hagyományos kivágásokhoz képest, a **TrOCR-base** esetében például az OBB alapú kivágás 16%-os javulást eredményezett.

3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

Images	Model	Annotation	Recognition rate (%)	CER	Recall (%)
50	TrOCR-base	non-obb	33.33	0.4607	46.35
50	TrOCR-base	obb	84.31	0.0308	96.92
50	TrOCR-small	non-obb	3.92	0.8539	12.08
50	TrOCR-small	obb	80.39	0.0504	92.72
50	Tesseract	non-obb	0.00	0.9101	3.93
50	Tesseract	obb	52.94	0.2857	64.15
50	EasyOCR	non-obb	19.61	0.5506	31.46
50	EasyOCR	obb	43.14	0.3221	57.14

3.6. táblázat. Szélsőségesen döntött rendszámok

Az alap annotációról az OBB-alapú megközelítésre való áttérés szükségességének szemléltetése érdekében egy különálló adathalmazt is létrehoztunk, amely kizárólag szélsőséges pozícióban elhelyezkedő, azaz jelentősen ferde vagy döntött szögben látható rendszámokat tartalmaz. A cél az volt, hogy olyan esetekben is értékelni lehessen a különböző OCR modellek teljesítményét, amelyekben a hagyományos, tengelyekkel párhuzamos határolókeretek pontatlanabb kivágást eredményeznek, ezáltal potenciálisan rontva a karakterfelismerés pontosságát. A 3.6. táblázat ezen az adathalmazon mért eredményeket mutatja. Megfigyelhető, hogy a **TrOCR-base** OBB alapú kivágások esetén 84.31%-os felismerési arányt ért el, 0.0308-as CER értékkel és 96.92%-os Recall értékkel, amelyek egyértelműen a legjobb eredménynek bizonyultak a vizsgált modellek közül. Különösen szembetűnő, hogy az OBB alapú kivágások ferde rendszámok esetén még nagyobb előnyt biztosítanak a hagyományos kivágáshoz képest, a **TrOCR-base** esetében ez 51%-os javulást jelent a felismerési arányban.

A 3.5. és a 3.6. táblázatok alapján egyértelműen kimondható, hogy a TrOCR jobb eredményt produkál úgy szélsőséges, mint átlagos pozíciójú rendszámok esetén, mint a többi tesztelt modell; a **TrOCR-base** modell körülbelül 4%-al teljesít jobban jelentősen döntött szögben található rendszámok leolvasásában, mint a kisebb verziója, a **TrOCR-small**, míg átlagos elhelyezkedésű rendszámok esetén körülbelül 7%-al.

A 3.7. táblázatban megfigyelhető, hogy Raspberry Pi 5-ön a **TrOCR-base** modell átlagosan **6.67** másodpercet igényel egyetlen rendszám leolvasásához, ezzel szemben a **TrOCR-small** átlagosan **1.39** másodpercet. Figyelembe véve, hogy a két modell pontossága között minimális a különbség, a közel valós idejű alkalmazhatóság szempontjából a **trocr-small-printed** modell alkalmazása mellett döntöttünk. Viszonyítási alapként a 3.8. táblázat az azonos méréseket laptopon mutatja, ahol a **TrOCR-small** átlagosan **0.32** másodperc alatt olvas le egy képet.

3. FEJEZET: AUTOMATIKUS RENDSZÁMLEOLVASÁS RASPBERRY PI-N

Model	Mean (s)	Std (s)
TrOCR-base	6.67	0.27
TrOCR-small	1.39	0.36
TesseractOCR	0.15	0.03
EasyOCR	4.99	0.07

3.7. táblázat. Egy rendszám leolvasásához szükséges idő Raspberry Pi 5-ön (5 futtatás átlaga és szórása).

Model	Mean (s)	Std (s)
TrOCR-base	2.09	0.06
TrOCR-small	0.32	0.02
TesseractOCR	0.16	0.02
EasyOCR	0.32	0.04

3.8. táblázat. Egy rendszám leolvasásához szükséges idő laptopon, viszonyítási alapként (5 futtatás átlaga és szórása).

3.6. Pipeline lépései a gyakorlatban

3.6.1. Rendszámok felismerése

A kamera képkockáit folyamatosan beolvassuk, és minden egyes képkockán inferenciát futtatunk a betanított, exportált **yolo11n-obb** modellel. A modell minden detektált rendszámhoz visszaadja az oriented bounding box négy sarokpontjának koordinátáit, valamint a detektálás megbízhatóságát (*confidence score*). Az összes detektált rendszám köré keret kerül kirajzolásra, függetlenül a megbízhatósági értéktől. Amennyiben a confidence értéke nem éri el a beállított küszöbértéket (**0.80**, avagy **80%**), a rendszám nem kerül feldolgozásra.



3.3. ábra. Detektált rendszámok.

3.6.2. Detektált rendszámok kivágása

A küszöbértéket meghaladó detektálások esetén a rendszám kivágása perspektíva transzformáció segítségével történik: a négy sarokpont alapján egy egyenes, vízszintes képet kapunk a rendszámról. Ezt követően a kivágott kép orientációja is szükségszerűen korrigálásra kerül: amennyiben a kép magasabb mint amilyen széles, 90 fokkal elforgatásra kerül, majd a kék EU-sáv pozíciója alapján meghatározzuk, hogy a rendszám helyes vagy fordított irányban szerepel-e, ha nem, akkor 180 fokkal elforgatjuk.



3.4. ábra. Kivágott rendszám 1.



3.5. ábra. Kivágott rendszám 2.

3.6.3. Rendszámok olvasása

A kivágott és orientált rendszámképet a **trocr-small-printed** modell dolgozza fel. A képet RGB formátumba konvertáljuk, majd a TrOCRProcessor előkészíti a modell számára. A modell által generált tokensorozatot visszaalakítjuk szöveggé, az így kapott nyers szövegen ezután utófeldolgozást (*post-processing*): eltávolításra kerülnek a nem alfanumerikus karakterek, alkalmazásra kerül a megyekód-szűrő, a rendszám szerkezeti javítása, valamint a hosszkorlátozás.

3.6.4. Olvasott rendszámok fájlba való logolása

Logolás előtt minden esetben leellenőrizzük az olvasott rendszám-érték helyességét, vizsgáljuk, hogy betartja-e a romániai rendszám formai kritériumait. Érvényes rendszám esetén egy további feltétel is teljesülnie kell: az adott rendszám legutóbbi logolása óta legalább **2 másodpercnek** kell eltelnie, ezzel elkerülve ugyanazon rendszám ismételt rögzítését. Ha mindkét feltétel teljesül, a rendszám időbélyeggel, a nyers OCR szöveggel és a detektálási confidence értékkel együtt kerül egy .txt fájlba, a következő formátumban:

```
2026-04-06T19:03:43 | raw=SJ 19 DAC -> SJ19DAC | det_conf=0.88
```

```
2025-04-06T19:03:44 | raw= SJ 66KRS -> SJ66KRS | det_conf=0.88
```

Összefoglaló

A dolgozat során létrehoztunk egy közel valós idejű, beágyazott rendszámfelismerő rendszert, amely képes videó stream-ről romániai rendszámtáblákat detektálni és leolvasni. A megvalósítás során leggyakrabban használt könyvtárak az **Ultralytics**, az **OpenCV**, valamint a **Hugging Face Transformers** voltak, melyeket Python programozási nyelv segítségével hívtunk meg.

A rendszer két fő fázisban működik: a rendszámtábla detektálása, illetve a detektált és kivágott képrészleten a szöveg leolvasása.

A rendszámtáblák detektálására egy mély tanulást alkalmazó módszert használtunk, a **yolo11n-obb** modellt, amelyet kizárólag saját, általunk gyűjtött és annotált adathalmazon tanítottunk be. Az oriented bounding box formátum alkalmazása lehetővé tette a ferde, döntött szögben elhelyezkedő rendszámok pontos keretezését is, ami hagyományos bounding box esetén nem lett volna megoldható. A modellt NCNN formátumra exportáltuk, hogy ARM alapú hardveren is hatékonyan fusson.

A szöveg leolvasására négy különböző OCR modellt hasonlítottunk össze: **TrOCR-base**, **TrOCR-small**, **TesseractOCR** és **EasyOCR**. Az összehasonlítást mind általános, mind szélsőségesen ferde rendszámokat tartalmazó adathalmazon elvégeztük. Az eredmények alapján a TrOCR modellek mindkét esetben jelentősen felülmúlták a többi megoldást. A sebesség és pontosság közötti kompromisszum figyelembevételével a **TrOCR-small** modell mellett döntöttünk, amely Raspberry Pi 5-ön átlagosan **1.39** másodperc alatt olvas le egyetlen rendszámot.

Kijelenthetjük, hogy a megoldásunk romániai rendszámok közel valós idejű felismerésére alkalmas. Ugyanakkor van néhány eset, melyre a rendszerünk nem ad kielégítő eredményt. Rendkívül rossz fényviszonyok, erősen szennyezett vagy részben takart rendszámtáblák, illetve a tanítási adathalmazban alulreprezentált szögek esetén a detektálás pontossága csökkenhet.

Eredményeinket tovább lehetne javítani az adathalmaz bővítésével, különösen éjszakai és gyenge megvilágítású képekkel, valamint a TrOCR modell romániai rendszámokra történő finomhangolásával (*fine-tuning*).

Irodalomjegyzék

- [1] URL: https://numpy.org/doc/stable/user/absolute_beginners.html (elérés dátuma 2025. 11. 18.).
- [2] URL: <https://docs.pytorch.org/tutorials/beginner/basics/intro.html> (elérés dátuma 2025. 11. 18.).
- [3] URL: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html (elérés dátuma 2026. 05. 08.).
- [4] Dr. Garima Sinha Ahmed Nur Ali Dr. Deepak K Sinha. „A Comprehensive Review of Automatic Number Plate Recognition”. *International Journal of Scientific Development and Research* 8.6 (2023). URL: <https://www.ijedr.org/papers/IJEDR2306140.pdf>.
- [5] Sayyed Humaira Ali. „A Review On Object Detection Techniques: From Traditional Methods To Deep Learning Approaches”. *INTERNATIONAL JOURNAL OF CREATIVE RESEARCH THOUGHTS (IJRCT)* (2025).
- [6] Hangbo Bao és mások. „BEiT: BERT Pre-Training of Image Transformers”. *ICLR 2022*. 2022. ápr. URL: <https://www.microsoft.com/en-us/research/publication/beit-bert-pre-training-of-image-transformers/>.
- [7] Herbert Bay, Tinne Tuytelaars és Luc Van Gool. „SURF: Speeded Up Robust Features”. *European Conference on Computer Vision*. 2006. URL: <https://api.semanticscholar.org/CorpusID:461853>.
- [8] Christopher Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. jan. URL: <https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/>.
- [9] Alexey Bochkovskiy, Chien-Yao Wang és Hong-Yuan Mark Liao. „Yolov4: Optimal speed and accuracy of object detection”. *arXiv preprint arXiv:2004.10934* (2020).
- [10] Michael Chen. „What Is Machine Learning?”. (2024). URL: <https://www.oracle.com/uk/artificial-intelligence/machine-learning/what-is-machine-learning/>.

IRODALOMJEGYZÉK

- [11] Atul Patel Chirag Patel Dipti Shah. „Automatic Number Plate Recognition System (ANPR): A Survey”. *International Journal of Computer Applications* 69.9 (2013. máj.), 21–33. oldal. ISSN: 0975-8887. DOI: [10.5120/11871-7665](https://doi.org/10.5120/11871-7665). URL: <https://ijcaonline.org/archives/volume69/number9/11871-7665/>.
- [12] „Computer vision recognizes objects, people, and patterns”. (). URL: <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-computer-vision>.
- [13] N. Dalal és B. Triggs. „Histograms of oriented gradients for human detection”. *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*. 1. kötet. 2005, 886–893 vol. 1. DOI: [10.1109/CVPR.2005.177](https://doi.org/10.1109/CVPR.2005.177).
- [14] Daswin De Silva és mások. „Toward Intelligent Industrial Informatics: A Review of Current Developments and Future Directions of Artificial Intelligence in Industrial Applications”. *IEEE Industrial Electronics Magazine* 14.2 (2020), 57–72. oldal. DOI: [10.1109/MIE.2019.2952165](https://doi.org/10.1109/MIE.2019.2952165).
- [15] Ankan Ghosh. „YOLO11: Redefining Real-Time Object Detection”. (2024). URL: <https://learnopencv.com/yolo11/> (elérés dátuma 2026. 04. 16.).
- [16] V.K Govindan és A.P Shivaprasad. „Character recognition — A review”. *Pattern Recognition* 23.7 (1990), 671–683. oldal. ISSN: 0031-3203. DOI: [https://doi.org/10.1016/0031-3203\(90\)90091-X](https://doi.org/10.1016/0031-3203(90)90091-X). URL: <https://www.sciencedirect.com/science/article/pii/003132039090091X>.
- [17] Alex Graves és mások. „Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks”. *Proceedings of the 23rd international conference on Machine learning* (2006). URL: <https://api.semanticscholar.org/CorpusID:9901844>.
- [18] Sepp Hochreiter és Jürgen Schmidhuber. „Long Short-Term Memory”. *Neural Computation* 9.8 (1997), 1735–1780. oldal. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [19] Jim Holdsworth. „What is optical character recognition (OCR)?”: (2024). URL: <https://www.ibm.com/think/topics/optical-character-recognition> (elérés dátuma 2026. 05. 05.).
- [20] ANPR International. *History of ANPR*. 2021. URL: <http://www.anpr-international.com/history-of-anpr/> (elérés dátuma 2026. 05. 21.).
- [21] Glenn Jocher és Jing Qiu. *Ultralytics YOLO11*. 11.0.0. verzió. 2024. URL: <https://github.com/ultralytics/ultralytics>.
- [22] Rahima Khanam és Muhammad Hussain. „Yolov11: An overview of the key architectural enhancements”. *arXiv preprint arXiv:2410.17725* (2024).

IRODALOMJEGYZÉK

- [23] Yann LeCun és Geoffrey Hinton. „Deep Learning”. *Nature* 521 (2015. máj.), 436–44. oldal. DOI: [10.1038/nature14539](https://doi.org/10.1038/nature14539).
- [24] Minghao Li és mások. „TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models”. *Proceedings of the AAAI Conference on Artificial Intelligence* 37.11 (2023. jún.), 13094–13102. oldal. DOI: [10.1609/aaai.v37i11.26538](https://doi.org/10.1609/aaai.v37i11.26538). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/26538>.
- [25] Yinhan Liu és mások. „Roberta: A robustly optimized bert pretraining approach”. *arXiv preprint arXiv:1907.11692* (2019).
- [26] David G. Lowe. „Distinctive Image Features from Scale-Invariant Keypoints”. *International Journal of Computer Vision* 60 (2004), 91–110. oldal. URL: <https://api.semanticscholar.org/CorpusID:174065>.
- [27] Lubna, Naveed Mufti és Syed Afaq Ali Shah. „Automatic Number Plate Recognition: A Detailed Survey of Relevant Algorithms”. *Sensors (Basel, Switzerland)* 21 (2021). URL: <https://api.semanticscholar.org/CorpusID:233457356>.
- [28] Jacob Maurel Ph.D és Eda Kavlakoglu. *What is object detection?* 2024. URL: <https://www.ibm.com/think/topics/object-detection> (elérés dátuma 2025. 03. 26.).
- [29] qq_45688383. *YOLOv11 C2PSA Module Details*. Accessed: 2025. 2025. URL: https://blog.csdn.net/qq_45688383/article/details/145838096.
- [30] R. Smith. „An Overview of the Tesseract OCR Engine”. *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. 2. kötet. 2007, 629–633. oldal. DOI: [10.1109/ICDAR.2007.4376991](https://doi.org/10.1109/ICDAR.2007.4376991).
- [31] StopAndGoyyy. *YOLOv11 Network Architecture Details*. URL: <https://blog.csdn.net/StopAndGoyyy/article/details/143169639> (elérés dátuma 2026. 06. 11.).
- [32] Øivind Due Trier, Anil K. Jain és Torfinn Taxt. „Feature extraction methods for character recognition-A survey”. *Pattern Recognit.* 29 (1996), 641–662. oldal. URL: <https://api.semanticscholar.org/CorpusID:205015030>.
- [33] Ultralytics. „COCO Dataset”. *Ultralytics YOLO Docs* (2023). URL: <https://docs.ultralytics.com/datasets/detect/coco/>.
- [34] Ultralytics. „Mean Average Precision (mAP)”. *Ultralytics YOLO Docs* (). URL: <https://www.ultralytics.com/glossary/mean-average-precision-map>.
- [35] Ultralytics. „Performance Metrics”. *Ultralytics YOLO Docs* (2024). URL: <https://docs.ultralytics.com/models/yolo11/#performance-metrics>.

IRODALOMJEGYZÉK

- [36] Ultralytics. „Ultralytics YOLO NCNN Export”. (2024). URL: <https://docs.ultralytics.com/integrations/ncnn> (elérés dátuma 2026. 05. 20.).
- [37] Ultralytics. „Ultralytics YOLO11”. *Ultralytics YOLO Docs* (2024). URL: <https://docs.ultralytics.com/models/yolo11/#overview>.
- [38] Ultralytics. „Ultralytics YOLO11”. *Ultralytics YOLO Docs* (2024). URL: <https://docs.ultralytics.com/models/yolo11/#supported-tasks-and-modes>.
- [39] Ultralytics. „YOLO: A Brief History”. *Ultralytics YOLO Docs* (2023). URL: <https://docs.ultralytics.com/#yolo-a-brief-history>.
- [40] Vladimir Vapnik. „Support-vector networks”. *Machine learning* 20 (1995), 273–297. oldal.
- [41] Ashish Vaswani és mások. „Attention is all you need”. *Advances in neural information processing systems* 30 (2017).
- [42] Abirami Vina. „Using Ultralytics YOLO11 for automatic number plate recognition”. (2024). URL: <https://www.ultralytics.com/blog/using-ultralytics-yolo11-for-automatic-number-plate-recognition>.